DOCUMENTATION

Astra Gate Portable version from BobH

README

Astra-Gate-Portable · 2026-11-09

---

1 这个项目是什么

Astra Gate（上游 `239573049/Astra-Gate`）的非官方 Windows 便携版构建。

- 上游：`239573049/Astra-Gate`（https://github.com/239573049/Astra-Gate）
- 许可证：MIT，与上游同一许可证条款
- 平台：Windows x64；ARM64 构建可手动生成
- 性质：非官方便携打包；本仓库不是源码 fork

本仓库不包含 Astra Gate 源码。每次发布都从上游 release tag 构建：浅克隆该 tag，叠加打包脚本，然后执行上游的构建命令。详见“构建流程”。

> 非官方构建。便携版本身的问题请在本仓库反馈；产品问题请找上游。

2 下载与运行

从本仓库的 Releases 页面下载最新的 `Astra-<version>-win32-x64-portable.zip` 和对应的 `.sha256` 文件。

1. 校验 SHA-256（见下文）。
2. 解压到任意目录。
3. 运行 `Astra.exe` 。

不需要安装程序、管理员权限、Node.js 或 .NET 运行时。应用默认使用上游的数据目录 `%USERPROFILE%\.astra` ；如果希望数据跟着程序目录走，可以在启动前设置上游支持的 `ASTRA_HOME` 环境变量。二进制没有代码签名，Windows SmartScreen 可能提示。

3 包内容

ZIP 包含：

- `Astra.exe`：Avalonia 桌面端，Native AOT。
- `resources/server/`：`astra-server.exe`、原生依赖和 Web 控制台。
- 来自上游 tag 的 `LICENSE` 和 `THIRD_PARTY_NOTICES.md`。

每次 Release 还提供：

- `Astra-<version>-win32-<arch>-portable.zip.sha256`
- `build-info.json`：上游 tag、RID、大小、SHA-256 和构建记录。

4 校验下载

`.sha256` 文件使用标准 `<sha256>  <filename>` 格式。

```
Get-FileHash .\Astra-<version>-win32-x64-portable.zip -Algorithm
SHA256
Get-Content .\Astra-<version>-win32-x64-portable.zip.sha256
```

有 `sha256sum` 的系统可以运行：

```
sha256sum -c Astra-<version>-win32-x64-portable.zip.sha256
```

release workflow 在发布前会校验一次，并把 hash 写进 Release notes。

4.1 Scoop

校验和资产的文件名就是 `<zip>.sha256`，Scoop manifest 可以直接使用：

```
"hash": {
  "url": "$url.sha256"
}
```

ARM64 资产按需调整 manifest 的 URL。

5 发布

稳定的上游 release 每 6 小时自动检查一次。Release tag 使用上游 tag 加前缀：

```
portable-<upstream tag>    # 例如 portable-v0.14.3100
```

预览版会标记为 prerelease，不会自动构建。定时构建针对 `win-x64`；`win-arm64` 构建需要手动生成。

6 从源码构建

需要：

- Windows 10/11，x64 或 arm64。
- Git、PowerShell 5+。
- Node.js 24、pnpm 9.15.0。
- .NET SDK `11.0.100-rc.3.26522.148`（上游 `global.json` 固定的版本）。

使用 mise（`mise install` 只安装 Node.js 和 pnpm；.NET SDK 需单独安装）：

```
mise install
mise run portable -- --tag v0.14.3100
# 最新稳定版
mise run portable
```

不使用 mise：

```
node wrapper/build-portable.mjs --tag v0.14.3100 --rid win-x64
node wrapper/build-portable.mjs --tag v0.14.3100 --rid win-arm64
```

常用检查：

```
# 只克隆、叠加、校验，不执行重构建
node wrapper/build-portable.mjs --tag v0.14.3100 --prepare-only

# 查看最新稳定版 tag
node wrapper/latest-upstream-tag.mjs

# 校验 mise 任务和脚本语法
mise tasks validate
node --check wrapper/build-portable.mjs
node --check wrapper/latest-upstream-tag.mjs
node --check scripts/package-portable.mjs
```

`--prepare-only` 可以在非 Windows 主机上运行；完整构建需要 Windows，因为 Windows Native AOT 发布和 ZIP 步骤依赖 Windows 工具链。完整构建大约需要

30–60 分钟。修改 构建 wrapper 的 pull request 会在 GitHub Actions 中运行同样的构建。

7 构建流程

1. 解析上游 tag：显式 `--tag`，或从 GitHub API 取最新稳定版。
2. 浅克隆 `https://github.com/<UPSTREAM>` 到 `.upstream/`（已被 gitignore）。
3. 校验 tag 与 `cli/package.json` 的版本一致，并确认上游仍有本仓库依赖的打包脚本。
4. 把 `scripts/package-portable.mjs` 复制进 checkout，并按需应用补丁。
5. 执行与上游 release 相同的构建步骤：pnpm 安装、Web 构建、服务端 Native AOT 发布、
6. 桌面端 Native AOT 构建/发布，然后运行服务端插件自检和桌面端 `--smoke`。
7. 运行 `scripts/package-portable.mjs`：摆放 `Astra.exe` 和服务端，生成 ZIP 与 SHA-256，
8. 解压 ZIP 并校验布局与文件大小。
9. 写入 `build-info.json`（上游 tag、RID、校验和、大小、workflow run）。

产物位于 `out/portable/<rid>/`：

```
Astra-<version>-win32-<arch>-portable.zip
Astra-<version>-win32-<arch>-portable.zip.sha256
build-info.json
```

8 仓库结构

只有 `scripts/package-portable.mjs` 使用与上游完全一致的路径；`scripts/` 目录可以 原样复制或合并进上游 checkout，不需要改路径。其余脚本都放在 `wrapper/`，刻意位于 上游目录树之外。

路径	用途	可提交上游？
scripts/package-portable.mjs	在完整上游 checkout 内生成便携 ZIP 的脚本。	是
wrapper/build-portable.mjs	Wrapper 专用编排：解析 tag、克隆、叠加脚本和补丁、构建、打包。	否
wrapper/latest-upstream-tag.mjs	通过 GitHub API 读取最新稳定版 tag。	否
.mise.toml	工具版本和本地任务。	否
.github/workflows/portable-ci.yml	PR/push 构建验证，不发布 Release。	否

路径	用途	可提交上游？
.github/workflows/portable-release.yml	定时/手动检查上游、构建并发布 Release。	否
UPSTREAM	一行 `owner/repo`。	否
patches/	可选的源码补丁；真正需要时才创建。	否
LICENSE	本 wrapper 的 MIT 许可证，条款与上游相同。	否
README.md, AGENTS.md	文档。	否

9 向上游贡献

目前只有一个文件可以作为 1:1 PR：

本仓库路径	上游路径	用法
scripts/package-portable.mjs	scripts/package-portable.mjs	原样复制；单独 commit；向上游 `main` 开 PR。

其他文件都不能直接推向上游：`wrapper/**`、`.github/workflows/**`、`.mise.toml`、`UPSTREAM`、`README.md`、`AGENTS.md`、`LICENSE` 都是 wrapper-only。根目录 `LICENSE` 是 wrapper 的许可证副本；ZIP 内的上游许可证来自构建时的上游 tag。`scripts/package-portable.mjs` 只使用上游已有的 `scripts/rid-map.mjs` 和 `scripts/pack-platform.mjs`，可以在干净的上游 checkout 中运行。

导出到 fork 分支：

```
git switch -c feat/portable-packaging upstream/main
Copy-Item <本仓库路径>/scripts/package-portable.mjs scripts/package-
portable.mjs
git add scripts/package-portable.mjs
git commit -m "feat: add portable packaging script"
```

本仓库中的路径已经是上游路径，不需要做路径转换。wrapper workflow、mise 配置和文档 不要放进上游 PR 分支。GitHub 没有“某路径只能出现在 PR 里”的配置；边界来自仓库、 分支和 commit。

建议先开 issue 问上游：便携 ZIP 是否受欢迎、脚本应放在哪个路径。

10 许可证

本仓库使用 MIT，许可证条款与上游 Astra Gate 相同。根目录 `LICENSE` 的版权行是本 wrapper 的维护者。便携 ZIP 内的 `LICENSE` 则是从所构建的上游 tag 复制的原始上游许可证（保留上游版权声明），并同时包含 `THIRD_PARTY_NOTICES.md` ；ZIP 重新分发 上游二进制，所以这些文件必须随包提供。

上游：https://github.com/239573049/Astra-Gate

11 限制

- 仅 Windows；这里没有 macOS/Linux 便携构建。
- 没有安装器集成、自动更新集成或代码签名。
- 数据目录默认不迁移；需要自包含目录时使用 `ASTRA_HOME` 。
- 预览版不会被自动发现。